المطعم: الحل والمناقشة

سنمشي خطوة خطوة: ما هي الصفوف؟ ماذا في كل صف من خصائص وتوابع؟ وكيف ترتبط ببعضها؟ ونسأل عند كل علاقة: **ليش هذه العلاقة وليست غيرها؟**

1 حلّل القصة: ما هي الصفوف؟

نلوّن الكلمات المهمة: **أزرق = صف** · **أخضر = معلومة (خاصية)** · **أحمر = عمل (تابع)** .

في **المطعم** **أصناف** لكل منها **اسم** و **سعر** ، و **زبائن** و **موظفون** لكل منهم **اسم** و **جوال** و **إيميل** .

الزبون **يدفع** ، والموظف **يستلم راتباً** ، والموصّل **يوصّل الطلب** ، والطلب **يضيف عنصراً** و **يحسب المجموع**

القاعدة: الاسم الذي له معلومات خاصة به يصبح **صفاً** (مثل الصنف والزبون والطلب). والاسم الذي هو مجرد معلومة يصبح **خاصية** (مثل الاسم والسعر). والفعل يصبح **تابعاً** (مثل `pay` و `deliver` و `total`).

جدول الصفوف: ماذا في كل صف؟

الصف	الخصائص (معلومات)	التوابع (أعمال)
`MenuItem` صف مجرد	`name, price`	`getName()` · `getPrice()` · `setPrice(price)`
`Meal` وجبة	`grams`	`toString()`
`Drink` مشروب	`ml`	`toString()` · `priceAfterDiscount()`
`Dessert` حلوى	`calories`	`toString()` · `priceAfterDiscount()`
`Discountable` واجهة	`-`	`priceAfterDiscount()`
`Person` صف مجرد	`name, phone, email`	`getName()` · `getPhone()` · `getEmail()` · `setEmail(email)`
`Customer` زبون	`address, points`	`getAddress()` · `getPoints()` · `isVip()` · `toString()` · `pay(amount)`
`Employee` صف مجرد	`salary`	`setSalary(salary)` · `salary()`
`Chef` شيف	`-`	`toString()`
`Driver` موصّل	`-`	`toString()` · `deliver(customer, order)`
`Payer` واجهة	`-`	`pay(amount)`
`Payable` واجهة	`-`	`salary()`
`OrderItem` عنصر الطلب	`menuItem, quantity`	`getMenuItem()` · `getQuantity()` · `subtotal()`
`Order` طلب	`customer, items`	`getCustomer()` · `addItem(menuItem, quantity)` · `getItems()` · `total()`

و `Main` فيه `main` فقط: يقرأ البيانات ويبني الأغراض ويطبع النتائج.

> **ماذا يعني `toString()` ؟ تابع موجود في كل صف في Java (يأتي من `Object`) يرجع حملة تصف الغرض.**
> **نكتبه من جديد بـ `@Override` فتصبح `System.out.println(item)` تطبع وصفاً جميلاً بدل `Meal@1b2c3d` .**

كيف نقرر؟ من القصة إلى الصفوف

نمشي بخمس خطوات، ونسأل سؤالاً واحداً في كل خطوة:

الخطوة	السؤال	في المطعم
1	ما **الأسماء** في القصة؟ (مرشحة لتصبح صفوفاً)	صنف، وجبة، مشروب، حلوى، زبون، موظف، شيف، موصّل، طلب، عنصر
2	أيّها مجرد **معلومة** عن غيره؟ (يصبح خاصية)	الاسم، السعر، الجوال، الإيميل، العنوان، الراتب: خصائص وليست صفوفاً
3	هل بين اسمين علاقة **"هو"**؟ (وراثة)	الوجبة هي صنف، والشيف هو موظف، والموظف هو شخص
4	ما **المشترك** بين الأبناء؟ (يذهب إلى الأب)	الاسم والسعر للأصناف، والاسم والجوال والإيميل للأشخاص
5	هل شيء **"يستطيع"** أن يفعله بعضهم دون بعض؟ (واجهة)	المشروب والحلوى يُخفَّضان، الزبون يدفع، الموظف يُدفع له

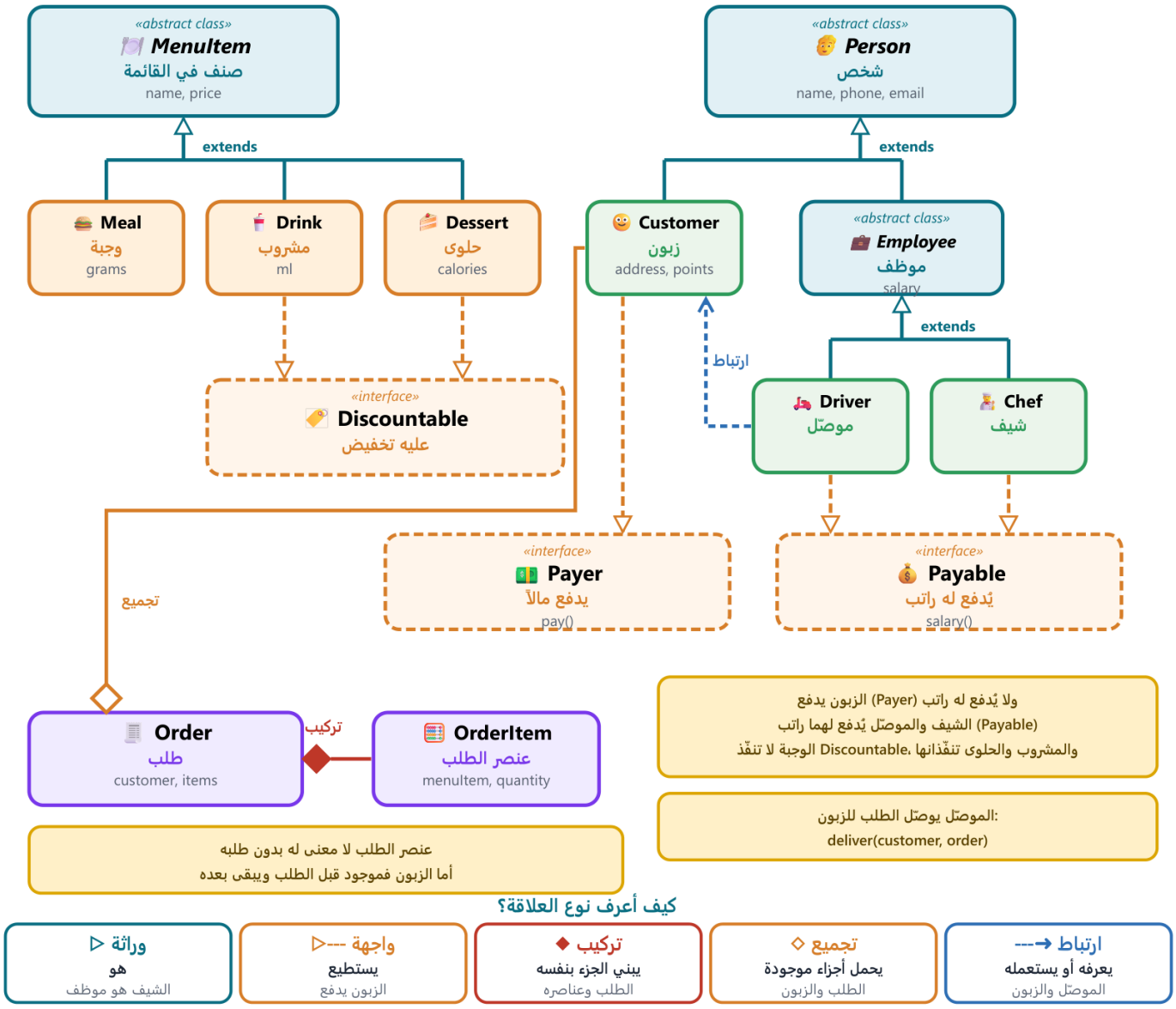
ما الفرق بين `MenuItem` و `OrderItem` ؟

الاسمان يتشابهان لكنهما **شيئان مختلفان تماماً**:

	`MenuItem` (صنف في القائمة)	`OrderItem` (عنصر الطلب)
ما هو؟	الشيء الذي يبيعه المطعم	عنصر في فاتورة الطلب: "أي صنف؟ وكم قطعة؟"
خصائصه	`name` , `price`	`item` (الصنف), `quantity` (الكمية)
مثال	بيتزا، سعرها 800$	"بيتزا × 2" في طلب نور
كم نسخة؟	واحدة لكل صنف في القائمة	واحدة لكل عنصر في كل طلب
متى يُبنى؟	مرة واحدة قبل أي طلب	عند كل `addItem`
متى يزول؟	يبقى بعد انتهاء الطلب	يزول مع طلبه

> مثال: ثلاثة طلبات كلها فيها بيتزا. عندنا **صنف بيتزا واحد** (`MenuItem`) و**ثلاثة عناصر طلب** (`OrderItem`) كل واحد بكميته. السعر يخص الصنف، والكمية تخص عنصر الطلب، ولهذا لا نضع الكمية داخل `MenuItem` .

2 مخطط الصفوف والعلاقات



3 أسئلة تحدد نوع العلاقة

لا تحفظ: اسأل هذه الأسئلة عن أي علاقة بين صفين **A** و**B**:

السؤال	إذا كان الجواب...
1) هل A **هو** نوع من B؟	نعم ← **وراثة**
2) هل A **يستطيع** أن يفعل شيئاً (عقد) يشترك فيه مع صفوف أخرى؟	نعم ← **واجهة**
3) هل B **جزء** من A؟	نعم ← اسأل أكثر ↓
3-أ) هل A أنشأ B بـ `new` داخله، ويزول B إذا زال A، ولا يشاركه غيره؟	نعم ← **تركيب** ◆
3-ب) هل B جاء من الخارج، ويبقى بعد زوال A، وقد يشاركه غيره؟	نعم ← **تجميع** ◇
4) B ليس جزءاً من A، لكن A **يعرفه** أو **يستعمله**؟	نعم ← **ارتباط** ---→

4 المناقشة: ليش هذه العلاقة؟

وراثة: ليش `Meal` يرث من `MenuItem` ؟ وليش `Chef` يرث من `Employee` الذي يرث من `Person` ؟

اختبار "هو": الوجبة **هي** صنف ✓، والمشروب **هو** صنف ✓. والشيف **هو** موظف ✓، والموظف **هو** شخص ✓، والزبون **هو** شخص ✓. يشتركون في الاسم والسعر، فرفعناها إلى الأب مرة واحدة.

لماذا الأب مجرد؟ لا يوجد في المطعم "صنف" فقط، بل وجبة أو مشروب أو حلوى. وكذلك لا يوجد "موظف" فقط، بل شيف أو موصّل.

لماذا فيه مستويان (`Person` ثم `Employee`)؟ لأن الراتب يخص الموظفين فقط، والزبون لا راتب له.

```
restaurant\Bad1.java:2: error: MenuItem is abstract; cannot be instantiated
public class Bad1 { public static void main(String[] a) { MenuItem m = new MenuItem("x", 1); } }
                                                                        ^
```

واجهة: ليش `Discountable` واجهة وليس تابعاً في `MenuItem` ؟ 💡

اللمبة: المشروب والحلوى عليهما تخفيض، لكن **الوجبة لا**. لو وضعنا `priceAfterDiscount()` في `MenuItem` لوجب على الوجبة كتابته بلا معنى. الواجهة تقول: **من يقبل التخفيض ينفّذها، ومن لا يقبل لا ينفّذها**.

ومحاولة تعامل الوجبة كأنها تقبل التخفيض لا تُترجم:

```
restaurant\Bad2.java:2: error: incompatible types: Meal cannot be converted to Discountable
public class Bad2 { public static void main(String[] a) { Discountable d = new Meal("pizza", 5, 300); } }
                                                                          ^
```

واجهة: ليش `Payer` و `Payable` ؟ (من يدفع ومن يُدفع له) 💡

الزبون **يدفع** ولا يُدفع له راتب، والشيف والموصّل **يُدفع لهما** راتب ولا يدفعان ثمن الطلب. كل واحد يأخذ العقد الذي يناسبه.

و `Payable` كتبناها مرة واحدة في `Employee` ، فورثها الشيف والموصّل معاً (الابن يرث الأب وواجهاته). أما استدعاء `salary()` على زبون فلا يُترجم:

```
restaurant\Bad3.java:2: error: cannot find symbol
public class Bad3 { public static void main(String[] a) { Customer c = new Customer("a", "1", "a@b", "x");
System.out.println(c.salary()); } }

^
  symbol:   method salary()
  location: variable c of type Customer
```

تركيب: ليش `OrderItem` ◆ `Order` ؟

الاختبار	عنصر الطلب
من أنشأه؟	الطلب بنفسه: `new OrderItem(...)` داخل `addItem`
ماذا لو حذفنا الطلب؟	يزول عنصره: "2 بيتزا" بدون الطلب لا معنى لها
هل يشاركه غيره؟	لا، كل عنصر في طلب واحد

إذن **تركيب**: "ملك لي ويموت معي". أما الصنف داخل العنصر فموجود في المطعم قبل الطلب، فلا يموت بموته.

تجميع: ليش `Customer` ◇ `Order` ؟ (ولماذا الزبون ليس تركيباً؟)

الاختبار	الزبون
من أنشأه؟	أُنشئ خارج الطلب (`new` في `main`) ثم وصل بالبنّاء
ماذا لو انتهى الطلب؟	يبقى الزبون موجوداً
هل يشاركه غيره؟	نعم: له طلبات كثيرة

إذن **تجميع**: "أحمله، وهو يعيش بدوني".

ارتباط: ليش `Customer` ---→ `Driver` ؟

الموصّل لا يملك الزبون ولا الطلب، ولا هما جزء منه. هو فقط **يوصّل له** في لحظة: `driver.deliver(customer, order)`. الزبون والطلب **وسيطان** للتابع وليسا خاصيتين في `Driver`. بعد انتهاء التابع لا يبقى أي رابط. هذا ارتباط "لحظي".

5 الكود: الصفوف المهمة وشرحها على الجنب

Employee.java

```java
package restaurant;

public abstract class Employee extends Person implements Payable {
    private int salary;

    public Employee(String name, String phone, String email, int salary) {
        super(name, phone, email);
        setSalary(salary);
    }

    public void setSalary(int salary) {
        this.salary = salary >= 0 ? salary : 0;
    }

    @Override
    public int salary() {
        return salary;
    }
}
```

وراثة: هو شخص + واجهة: يُدفع له

الراتب خاصية خاصة

تحقق: لا راتب سالب

ينفّذ بند العقد Payable

Drink.java

```java
package restaurant;

public class Drink extends MenuItem implements Discountable {
    private int ml;

    public Drink(String name, int price, int ml) {
        super(name, price);
        this.ml = ml;
    }

    @Override
    public String toString() {
        return "Drink: " + getName() + " (" + ml + " ml, " + getPrice() + "$)";
    }

    @Override
    public int priceAfterDiscount() {
        return getPrice() * 90 / 100;
    }
}
```

وراثة: هو صنف + واجهة: يُخفَّض

يصف نفسه بطريقته

تخفيض 10%

Order.java

```java
package restaurant;

import java.util.ArrayList;

public class Order {
    private Customer customer;
    private ArrayList<OrderItem> items = new ArrayList<>();
```

تجميع: الزبون يأتي من الخارج

تركيب: العناصر جزء من الطلب

```
    public Order(Customer customer) {
        this.customer = customer;
    }

    public Customer getCustomer() {
        return customer;
    }

    public void addItem(MenuItem menuItem, int quantity) {
        items.add(new OrderItem(menuItem, quantity));
    }

    public ArrayList<OrderItem> getItems() {
        return items;
    }

    public int total() {
        int sum = 0;
        for (OrderItem orderItem : items) {
            sum += orderItem.subtotal();
        }
        return sum;
    }
}
```

new داخل التابع = تركيب

Customer.java

```
package restaurant;

public class Customer extends Person implements Payer {
    private String address;
    private int points;

    public Customer(String name, String phone, String email, String address) {
        super(name, phone, email);
        this.address = address;
        this.points = 0;
    }

    public String getAddress() {
        return address;
    }

    public int getPoints() {
        return points;
    }

    public boolean isVip() {
        return points >= 20;
    }

    @Override
    public String toString() {
        return "Customer: " + getName() + " (" + getPhone() + ", " + getEmail() + ", " + address + ", " + points + " poin
```

وراثة: هو شخص + واجهة: يدفع

```
    }

    @Override
    public void pay(int amount) {
        System.out.println(getName() + " pays " + amount + "$");
        points += amount / 100;
    }
}
```

نقطة عن كل 100 دولار

20 نقطة فأكثر = مميز

Driver.java

```
package restaurant;

public class Driver extends Employee {

    public Driver(String name, String phone, String email, int salary) {
        super(name, phone, email, salary);
    }

    @Override
    public String toString() {
        return "Driver: " + getName() + " (" + getPhone() + ", " + getEmail() + ", salary " + salary() + "$)";
    }

    public void deliver(Customer customer, Order order) {
        System.out.println(getName() + " delivers to " + customer.getName() + " at " + customer.getAddress()
                + " (call " + customer.getPhone() + ") an order of " + order.total() + "$");
    }
}
```

ارتباط: الزبون والطلب وسيطان، ولا يُخزَّنان

باقي الصفوف في مجلد `code/src/restaurant`، وكلها بتعليقات عربية.

6 تتبّع الناتج

الأخضر ما يكتبه المستخدم بالكيبورد:

```
How many menu items?
4
Item 1: 1 = Meal, 2 = Drink, 3 = Dessert
1
Name:
Pizza
Price:
800
Grams:
300
Item 2: 1 = Meal, 2 = Drink, 3 = Dessert
2
Name:
Juice
Price:
150
Ml:
250
Item 3: 1 = Meal, 2 = Drink, 3 = Dessert
3
Name:
Cake
Price:
300
Calories:
400
Item 4: 1 = Meal, 2 = Drink, 3 = Dessert
2
Name:
Cola
Price:
100
Ml:
330
How many people?
4
Person 1: 1 = Customer, 2 = Chef, 3 = Driver
1
Name:
Nour
Phone:
0933111222
Email:
nour@mail.com
Address:
Mazzeh
Person 2: 1 = Customer, 2 = Chef, 3 = Driver
2
Name:
Hana
Phone:
0944333444
Email:
hana@mail.com
Salary:
5000
Person 3: 1 = Customer, 2 = Chef, 3 = Driver
3
Name:
Rami
Phone:
0955666777
Email:
rami.mail.com
Salary:
3500
Person 4: 1 = Customer, 2 = Chef, 3 = Driver
1
```

```
Name:
Omar
Phone:
0966888999
Email:
omar@mail.com
Address:
Kafarsouseh
======== Menu ========
Meal: Pizza (300 g, 800$)
Drink: Juice (250 ml, 150$)
Dessert: Cake (400 cal, 300$)
Drink: Cola (330 ml, 100$)
======== People ========
Customer: Nour (0933111222, nour@mail.com, Mazzeh, 0 points)
Chef: Hana (0944333444, hana@mail.com, salary 5000$)
Driver: Rami (0955666777, no email, salary 3500$)
Customer: Omar (0966888999, omar@mail.com, Kafarsouseh, 0 points)

======== Discounts ========
Juice: 150$ -> 135$
Cake: 300$ -> 240$
Cola: 100$ -> 90$
No discount: 1

======== Salaries ========
Hana: 5000$
Rami: 3500$
Total salaries: 8500$

======== Order ========
How many order items?
3
Item number (1 to 4):
1
Quantity:
2
Item number (1 to 4):
2
Quantity:
1
Item number (1 to 4):
3
Quantity:
1
Pizza x2 = 1600$
Juice x1 = 150$
Cake x1 = 300$
Total: 2050$

======== Payment and delivery ========
Rami delivers to Nour at Mazzeh (call 0933111222) an order of 2050$
Nour pays 2050$
Nour has 20 points, VIP: true
```

لاحظ: الوجبة لم تظهر في التخفيضات (No discount: 1)، والزبون لم يظهر في الرواتب، وإيميل الموصّل صار `no email` لأنه بلا `@` .

7 أخطاء شائعة

الخطأ	لماذا؟	الصحيح
نسيان `toString()` في صف ابن	تطبع `Meal@1b2c3d` بدل الوصف	اكتبه بـ `@Override` في كل صف
إنشاء `new OrderItem` في `main` وتمريره للطلب	صار العنصر يأتي من الخارج: انقلب إلى تجميع	استدع `order.addItem(...)`
وضع `priceAfterDiscount()` في `MenuItem`	الوجبة لا تقبل تخفيضاً، فتكتب تابعاً بلا معنى	واجهة `Discountable`
تخزين `Customer` كخاصية داخل `Driver`	صار ارتباطاً مخزّناً بينما المقصود توصيل لحظي	مرّره وسيطاً: `deliver(customer, ...)`
نسيان `instanceof` قبل التحويل إلى `Discountable`	الوجبة ليست `Discountable` فينهار البرنامج	`if (item instanceof Discountable)`

8 جواب سؤال الفهم (طلب 9)

الزوج	العلاقة	لماذا؟
`Order` و `OrderItem`	تركيب	الطلب يبني العناصر بـ `new` داخله، وتزول بزواله
`Order` و `Customer`	تجميع	الزبون أُنشئ خارج الطلب ويبقى بعده، وله طلبات كثيرة
`Driver` و `Customer`	ارتباط	الزبون وسيط للتابع `deliver` ولا يملكه الموصّل
`Chef` و `Employee`	وراثة	الشيف **هو** موظف
`Drink` و `Discountable`	واجهة	المشروب **يستطيع** أن يُخفَّض سعره (عقد)
`Customer` و `Payer`	واجهة	الزبون **يستطيع** أن يدفع (عقد)

"لماذا لم نضع تابع التخفيض في `MenuItem` ؟": لأن الوجبة لا تُخفَّض، فالواجهة تجعل من يستطيع فقط ينفّذ العقد.
"لماذا `Customer` لا ينفّذ `Payable` ؟": لأن الزبون لا يُدفع له راتب، هو من يدفع.

الملخص: وراثة = "هو" · واجهة = "يستطيع" · تركيب = "ملك لي ويموت معي" · تجميع = "أحمله وهو يعيش بدوني" · ارتباط = "أعرفه أو أستعمله فقط".

9 تمارين للتدريب

❓ **فكّر: سؤال 1:** لو أضفنا `Soup` (شوربة) صنفاً جديداً يقبل التخفيض، ماذا نفعل؟
الجواب: نجعله يرث من `MenuItem` وينفّذ `Discountable` . لا نغيّر أي حلقة قديمة (تعدد أشكال).

❓ **فكّر: سؤال 2:** لو أردنا أن تكون `Meal` تقبل التخفيض أيضاً، ماذا نفعل؟
الجواب: نكتب `implements Discountable` في `Meal` ونكتب `priceAfterDiscount()` فيها.

❓ **فكّر: سؤال 3:** صح أم خطأ: العلاقة بين `Order` و `Customer` تركيب لأن الطلب يحمل الزبون.
الجواب: خطأ. الزبون موجود قبل الطلب ويبقى بعده وله طلبات كثيرة، فهي تجميع.

❓ **فكّر: سؤال 4:** لماذا لا يخزّن `Driver` الزبون كخاصية؟
الجواب: لأن الموصّل لا يملك الزبون ولا هو جزء منه، يوصّل له لحظياً فقط، فيأتي الزبون وسيطاً للتابع.

10 أسئلة قد تخطر ببالك

هذه الأسئلة يسألها الطلاب عادة، وكل سؤال معه جوابه:

❓ **فكّر: 1) لماذا `OrderItem` صف مستقل؟ لماذا لا نضع `ArrayList<MenuItem>` داخل الطلب فقط؟**
لأن الطلب يحتاج لكل صنف **كمية**. وقائمة `MenuItem` وحدها لا تعرف أن البيتزا مطلوبة مرتين. فالعنصر (الصنف مع كميته) له معلومات خاصة به، وكل ما له معلومات خاصة يصبح صفاً.

❓ **فكّر: 2) لماذا لا نضع الكمية داخل `MenuItem` ؟**
لأن الكمية تتغير من طلب إلى آخر، بينما الصنف واحد. لو وضعنا الكمية في الصنف لصارت البيتزا "مطلوبة مرتين" في كل الطلبات معاً!

❓ **فكّر: 3) لماذا `OrderItem` تركيب مع الطلب، بينما `MenuItem` داخله ليس تركيباً؟**
العنصر **يُبنى داخل الطلب** ويزول معه. أما الصنف فجاء من القائمة، وبقي بعد انتهاء الطلب، ويشير إليه كثير من الطلبات. لو حذفنا الطلب، يبقى الصنف في القائمة.

❓ **فكّر: 4) كيف أعرف أن الشيء صف وليس خاصية؟**
اسأل: هل له معلومات خاصة به؟ **الزبون** له اسم وجوال وعنوان، فهو صف. أما **الاسم** فهو مجرد نص يخص الزبون، فهو خاصية.

❓ **فكّر: 5) ما الفرق بين `abstract class` و `interface` ؟ ومتى أستعمل كل واحد؟**

	صف مجرد	واجهة
تعني	"هو" (الشيف هو موظف)	"يستطيع" (المشروب يُخفَّض)
فيها خصائص؟	نعم (`name` و `price`)	لا
فيها كود جاهز؟	نعم، ومعها توابع مجردة	توابع بلا جسم فقط
كم أرث؟	أباً **واحداً** فقط	واجهات **كثيرة**

❓ **فكّر: 6) لماذا `Person` و `MenuItem` مجردان ولا يمكن عمل `new` لهما؟**
لأن لا يوجد في المطعم "شخص" فقط أو "صنف" فقط، بل زبون أو شيف، ووجبة أو مشروب. الصف المجرد وُجد ليجمع المشترك، وليس ليُبنى منه غرض.

❓ **فكّر: 7) لماذا لا نكتب `Customer` و `Employee` دون `Person` ؟**
لأننا سنكرر الاسم والجوال والإيميل في كل صف. بالوراثة نكتبها مرة واحدة في `Person` ، ولو أضفنا خاصية جديدة (مثل تاريخ الميلاد) نضيفها في مكان واحد.

❓ **فكّر: 8) لماذا `Employee` صف مجرد وليس واجهة؟**
لأن فيه **خاصية** (`salary`) وكوداً مشتركاً (`setSalary`). الواجهة لا تحمل خصائص ولا كوداً.

❓ **فكّر: 9) ولماذا `Payable` واجهة وليست صفاً؟**

لأن "يُدفع له راتب" قدرة قد تخص صفوفاً لا قرابة بينها لاحقاً (مثلاً متعاقد خارجي ليس موظفاً). والصف الواحد لا يرث إلا أباً واحداً، أما الواجهة فيمكن إضافتها بجانبه.

❓ **فكّر: 10) هل يمكن أن يرث صف من صفين؟**

لا في Java. لكن يمكنه `implements` أي عدد من الواجهات، مثل: `class Drink extends MenuItem implements Discountable, Vegetarian`.

❓ **فكّر: 11) لماذا نكتب `@Override` ؟**

لتقول للمترجم: "أنا أعيد كتابة تابع موجود في الأب أو في الواجهة". فإن أخطأت باسمه (مثل `tostring`) يعطيك خطأً بدل أن يمرّ بصمت.

❓ **فكّر: 12) لماذا الخصائص `private` ولا نجعلها `public` ؟**

لنتحكم بما يدخل إليها. مثال: `setPrice` ترفض السعر السالب، و `setEmail` ترفض الإيميل بلا `@` . لو كانت `public` لكتب أي أحد أي قيمة.

❓ **فكّر: 13) كيف تحمل `ArrayList<MenuItem>` وجبة ومشروباً وحلوى معاً؟**

لأن كل واحد منهم **هو** `MenuItem` . وكل صنف يطبع وصفه الخاص بسبب `toString()` المكتوب عنده. هذا هو تعدد الأشكال.

❓ **فكّر: 14) لماذا نحتاج `instanceof` وتحويلاً (`cast`) مع `Discountable` ؟**

لأن القائمة من نوع `MenuItem` والصنف العام لا يعرف `priceAfterDiscount()` . نسأل أولاً: "هل هذا الصنف يقبل التخفيض؟" (`instanceof`)، ثم نحوّله ونستعمله.

❓ **فكّر: 15) ما الفرق بين التركيب والتجميع بجملة واحدة؟**

التركيب: الكل يبني الجزء بـ `new` داخله ويزول معه. **التجميع:** الجزء يأتي من الخارج (بالبنّاء أو بتابع) ويبقى بعده.

❓ **فكّر: 16) لماذا `Order` يحمل `Customer` وليس العكس (الزبون يحمل طلباته)؟**

اتجاه العلاقة يتبع **الحاجة**: الطلب يحتاج زبوناً ليعرف لمن يُوصَّل، أما الزبون فلا يحتاج طلباته في هذا التمرين. ولو أردنا سجل طلبات للزبون لأضفناه لاحقاً، لكن ننتبه حتى لا يصبح كل صف يحمل الآخر.

❓ **فكّر: 17) لماذا `Driver` لا يحتفظ بالزبون كخاصية؟**

لأن الموصّل لا يملكه ولا هو جزء منه، يلتقيه لحظة التوصيل فقط. فيأتي الزبون **وسيطاً** للتابع `deliver` ، وينتهي الأمر بنهاية التابع. هذا هو الارتباط.

❓ **فكّر: 18) لماذا الزبون يدفع (`Payer`) والموظف يُدفع له (`Payable`)؟ ألا يكفي واجهة واحدة؟**

لأنهما عملان مختلفان لا يجتمعان في الصف نفسه: الزبون لا راتب له، والموظف لا يدفع ثمن الطلب. واجهتان تجعلان كل صف يأخذ العقد الذي يناسبه فقط.

❓ **فكّر: 19) هل يمكن أن يكون الشخص زبوناً وموظفاً معاً (شيف يطلب طعاماً)؟**

في تصميمنا لا، لأن الوراثة تعني "هو" دائماً. هذا سؤال متقدم: الحل أن نمثل "الدور" بصف مستقل (ستتعلمونه لاحقاً). في التمرين نعتبر كل شخص نوعاً واحداً.

❓ **فكّر: 20) من أين أبدأ لو أعطوني قصة مثل هذه من الصفر؟**

1) اقرأ القصة وضع خطاً تحت الأسماء. 2) ارم ما هو معلومة (يصبح خاصية). 3) ابحث عن "هو" (وراثة) ثم المشترك (الأب). 4) ابحث عن "يستطيع" (واجهة). 5) اسأل لكل زوج: هل يبنيه؟ هل يحمله؟ هل يستعمله فقط؟ 6) ارسم المخطط ثم اكتب الكود صفاً صفاً، وشغّل بعد كل صف.